Andrew Reed *reedas1* (*#94006807*)

CS121 Assignment 3 Milestone 1 Report

The group consists of one Software Engineering major, so provided metrics reflect the Algorithms and Data Structures Developer specifications.

Index Metrics

Metric	**Result**
Documents Indexed	55,393
Unique Tokens (Stemmed)	1,082,386
Partial Indexes Created	6

“Partial Indexes Created” refers to the number of partial indexes created *by default* on a given run of the `build_index` script (see README). The number of partial indexes created can be customized by the user, but defaults to creating one on-disk partial index per 10,000 indexed pages. See “Implementation Design” for more information.

Index File Sizes

File	**Size**
Document Metadata (`documents.jsonl`)	5,512.21 KB
Lexicon (`lexicon.json`)	54,533.36 KB
Postings (`postings.bin`)	148,836.29 KB
Total	208,881.87 KB

The total size of all index files is 208,881.87 kilobytes or about 203.99 megabytes. For the purposes of Milestone 1, however, the document metadata file is not necessary, so the total index size may more accurately be **203,369.65** kilobytes or or about **198.60** megabytes.

Implementation Design

All code used for the indexing process is found in the `search_engine/indexer` directory.

`document.py` defines the `Document` class, a lightweight dataclass used to represent information about parsed documents. This includes their assigned document ID, source URL (defragmented), raw content, encoding scheme, and source file system path. This data aligns with the structure of the provided JSON webpage data.

`reader.py` defines functions used for the discovery of documents in the raw data directory. `iter_documents`, which yields single documents at a time from the source directories for processing, is the main function used in the indexer pipeline.

`parser.py` defines the HTML parsing logic used between document discovery and tokenization. Content is parsed such that text within important elements—any child or direct text of `b`, `strong`, `title`, `h1`, `h2`, and `h3` tags—is stored as a subset of the general text content. Non-text elements are skipped. Parsing is performed with BeautifulSoup using the LXML parser, implemented similarly to Assignment 2's parsing logic but with fewer validation conditions.

`tokenizer.py` defines lightweight tokenization logic using a Python pattern. Stemming has been implemented with `nltk`'s Porter stemmer. The tokenizer `count_terms` function, which is its main outward-facing function, returns a Counter associating terms with their frequencies.

`builder.py` defines the core logic of adding a given document to an in-memory inverted index with `add_document_to_index`.

The heaviest file in terms of content is the `storage.py` file, which defines a number of functions used for storing partial index information on the disk. `write_partial_index` serializes all in-memory inverted index data to a formatted partial index file on the disk. The `merge_partial_indexes` function, by far the most complex, handles merging the given partial indexes into a lexicon and a postings file, enforcing ordering with a min-heap data structure.

`pipeline.py` connects the above functionality into a single index building function, `build_index`. It also defines the default file path information for the results of the indexing process.

The `build_index.py` script (found in the `scripts` directory) is used to run the full indexing process. See the README file for more information.